
User:

✅ 10. Parallel Streams

ChatGPT:

Here's a concise, **interview-ready explanation** of **Parallel Streams** in Java, perfect for Spring Boot or Microservices backend roles:

✅ 10. Parallel Streams

Definition:

Parallel streams split the source data into multiple substreams and process them **concurrently**, leveraging **multiple CPU cores**.

🔍 Syntax:

```
java
list.parallelStream().forEach(System.out::println);
```

🧠 Key Concepts:

Feature	Description
Parallelism	Utilizes ForkJoinPool.commonPool() by default (number of threads = CPU cores)
Splitting & Merging	Data is split → processed in parallel → combined into a single result
Stateless & Non-interfering	Operations must be stateless and not modify the source
Order	Parallel streams may not preserve order unless <code>.forEachOrdered()</code> is used

⚠️ When to Use:

- Large data sets
 - CPU-intensive tasks
 - When order doesn't matter or `.forEachOrdered()` is acceptable
-

🚫 When *Not* to Use:

- Small collections (parallelism overhead > gain)
- Stateful operations (e.g., modifying shared data)

- I/O-bound operations (threads wait rather than compute)
-

Example:

```
java
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);

int sum = nums.parallelStream()
    .mapToInt(n -> n * 2)
    .sum(); // May run in parallel
```

Pro Tip for Interview:

- Use `ForkJoinPool.commonPool()` sizing cautiously. Use `System.setProperty("java.util.concurrent.ForkJoinPool.common.parallelism", "4")` to control threads.
 - Prefer `.collect(Collectors.toList())` over `.forEach()` for predictable results.
-

Tricky Interview Question:

Q: *Can parallel streams be faster than normal loops?*

A: Yes, *if the operation is CPU-intensive, stateless, and run on a large dataset — otherwise, thread overhead may cause slower performance.*

Want to dive into `ForkJoinPool` or parallel vs sequential performance comparison next?
